Universidad Rafael Landívar
Facultad de Ingeniería
Inteligencia Artificial

TicketAI

Sistema de Clasificación Automática de Tickets

Integrantes del grupo

David Monje - 1019123
Sebastian Morales - 1057123
Raquel Albizures - 1122323

20 de abril de 2026

Índice

1. Introducción *4*

2. Definición del Problema y Objetivos *4*

2.1 Problema **4**

2.2 Objetivo General **4**

2.3 Objetivos Específicos **4**

3. Descripción del Dataset *5*

3.1 Fuente **5**

3.2 Características **5**

3.3 Distribución de Categorías **5**

3.4 Ejemplo de Datos **6**

4. Descripción del Preprocesamiento *6*

4.1 Pipeline de Preprocesamiento **6**

Paso 1. Eliminación de placeholders 6

Paso 2. Conversión a minúsculas 6

Paso 3. Eliminación de puntuación 6

Paso 4. Tokenización 6

Paso 5. Filtrado de stopwords 7

Paso 6. Stemming 7

4.2 Ejemplo Completo **7**

4.3 Herramientas Utilizadas **7**

4.4 Resultado del Preprocesamiento **7**

5. Explicación Matemática del Algoritmo *8*

5.1 Naive Bayes Multinomial **8**

5.2 Probabilidad A Priori (Prior) **8**

5.3 Bag of Words (BoW) **9**

5.4 Laplace Smoothing **9**

5.5 Suma de Logaritmos (Log-Sum) **10**

5.6 Conversión a Confianza (Softmax) **11**

6. Métricas de Evaluación *12*

6.1 Definiciones y Fórmulas **12**

6.2 Resultados Obtenidos **13**

6.3 Matriz de Confusión **13**

6.4 Análisis Crítico de la Matriz de Confusión **14**

7. Resultados de K-Folds Cross Validation *14*

7.1 Metodología **14**
7.2 Resultados por Fold **15**
7.3 Estadísticas Agregadas **15**
7.4 Análisis de Varianza **15**
8. Diagramas ***16***
8.1 Arquitectura de la Solución **16**
8.2 Diagrama de Casos de Uso **17**
8.3 Diagrama de Flujo General **18**
8.4 Diagrama de Componentes **19**
8.5 Diagrama de Secuencias. Clasificación de Ticket **20**
9. Evidencias de Funcionamiento ***20***
9.1 Página de inicio (Landing Page) 20
9.2 Creación de ticket 21
9.3 Resultado de clasificación 21
9.4 Cola de tickets 22
9.5 Detalle de ticket 22
9.7 Playground de clasificación 23
9.8 Dashboard de métricas 24
9.9 Matriz de confusión 25
9.10 Resultados K-Fold (Gráfico) 25
10. Conclusiones y Aprendizajes ***26***
10.1 Conclusiones **26**
10.2 Aprendizajes **26**
Anexo A. Stack Tecnológico Completo ***27***
Frontend **27**
Backend **27**
Infraestructura **28**
Anexo B. Guardado y Carga del Modelo ***28***
Anexo C. Instrucciones de Instalación y Ejecución ***29***
Opción 1. Docker Compose (recomendado) **29**
Opción 2. Desarrollo local **29**
Ejecutar tests **29**

1. Introducción

El presente documento describe el desarrollo de TicketAI, un sistema de clasificación automática de tickets de soporte al cliente basado en el algoritmo Naive Bayes Multinomial implementado desde cero. El sistema permite a los usuarios ingresar solicitudes de soporte que son automáticamente categorizadas y dirigidas al departamento correspondiente sin intervención humana.

El proyecto integra un motor de inferencia en Python (FastAPI) con una aplicación web moderna en Vue 3, desplegados mediante Docker Compose. El modelo fue entrenado con el dataset Bitext Customer Support (26,872 muestras, 11 categorías) y evaluado mediante validación cruzada K-Fold (K=5), obteniendo una accuracy promedio del 99.09%.

El énfasis del proyecto está en la comprensión e implementación manual de cada etapa del algoritmo: preprocesamiento de texto, construcción del vocabulario (Bag of Words), cálculo de probabilidades con Laplace Smoothing, uso de logaritmos para evitar underflow numérico, y evaluación rigurosa del modelo con métricas por clase y matriz de confusión.

2. Definición del Problema y Objetivos

2.1 Problema

Las mesas de ayuda reciben grandes volúmenes de solicitudes diarias que deben ser clasificadas manualmente por agentes humanos antes de ser dirigidas al departamento correcto. Este proceso es lento, propenso a errores y costoso. Se necesita un sistema que automatice la clasificación de solicitudes en categorías predefinidas con alta precisión.

2.2 Objetivo General

Desarrollar un sistema funcional de clasificación automática de tickets de soporte al cliente utilizando el algoritmo Naive Bayes Multinomial implementado desde cero, integrado con una aplicación web que permita la creación, clasificación en tiempo real y gestión de tickets.

2.3 Objetivos Específicos

1. Preprocesar datos textuales aplicando limpieza, tokenización, eliminación de stopwords y stemming.
2. Construir un vocabulario mediante Bag of Words a partir del corpus de entrenamiento.
3. Implementar manualmente el algoritmo Naive Bayes Multinomial, incluyendo cálculo de probabilidades a priori por clase, cálculo de verosimilitud con Laplace Smoothing para palabras no vistas, y uso de suma de logaritmos durante la inferencia para evitar underflow numérico.
4. Evaluar el modelo utilizando K-Folds cross validation (K=5) implementado manualmente.

5. Analizar e interpretar las métricas de evaluación: Precisión, Recall, F1-Score por clase, Accuracy global, Macro F1 y Matriz de Confusión.
6. Guardar y cargar el modelo entrenado desde un archivo persistente (pickle).
7. Desarrollar una interfaz web que simule un portal de soporte al cliente con roles diferenciados (Cliente, Agente, Administrador).
8. Integrar correctamente el backend (motor de inferencia) con el frontend (aplicación web).

3. Descripción del Dataset

3.1 Fuente

Nombre: Bitext Customer Support LLM Chatbot Training Dataset

Plataforma: HuggingFace

URL: https://huggingface.co/datasets/bitext/Bitext-customer-support-llm-chatbot-training-dataset

Formato de descarga: Parquet (5.8 MB) vía la API de HuggingFace

3.2 Características

Propiedad	Valor
Total de muestras	26,872
Número de categorías	11
Idioma	Inglés
Campo de texto	instruction
Campo de etiqueta	category
Etiquetas	Verificadas y semánticamente coherentes

3.3 Distribución de Categorías

Categoría	Cantidad	Porcentaje	Descripción
ACCOUNT	5,986	22.28%	Gestión de cuenta
ORDER	3,988	14.84%	Estado de ordenes
REFUND	2,992	11.13%	Reembolsos
INVOICE	1,999	7.44%	Facturacion
CONTACT	1,999	7.44%	Información de contacto
PAYMENT	1,998	7.43%	Métodos de pago
FEEDBACK	1,997	7.43%	Retroalimentacion
DELIVERY	1,994	7.42%	Estado de entregas
SHIPPING	1,970	7.33%	Envios
SUBSCRIPTION	999	3.72%	Suscripciones
CANCEL	950	3.54%	Cancelaciones

Observación: El dataset presenta un desbalance moderado. ACCOUNT tiene casi 6 veces más muestras que CANCEL. Esto se refleja en las probabilidades a priori del modelo y puede influir en la clasificación de textos ambiguos.

3.4 Ejemplo de Datos

instruction	category
I want to cancel my subscription {{Subscription ID}}	CANCEL
Where is my order? I placed it last week	ORDER
I need a refund for my damaged product	REFUND

Nota: Los patrones {{Placeholder}} (como {{Order ID}}, {{Account Number}}) son marcadores anonimizados que deben eliminarse durante el preprocesamiento.

4. Descripción del Preprocesamiento

Archivo: backend/app/naive_bayes/preprocessor.py

El preprocesamiento transforma texto crudo en una lista de tokens normalizados que el modelo puede procesar. Se implementó un pipeline de 6 pasos secuenciales.

4.1 Pipeline de Preprocesamiento

Paso 1. Eliminación de placeholders

Se utiliza una expresión regular para remover los marcadores {{...}} del dataset:

```
Regex: \{\{.*?\}\}
Entrada: "My order {{Order123}} is late"
Salida:  "My order  is late"
```

Paso 2. Conversión a minúsculas

```
Entrada: "My Order Is Late"
Salida:  "my order is late"
```

Paso 3. Eliminación de puntuación

Se remueven todos los caracteres de puntuación utilizando string.punctuation.

```
Entrada: "help! can't login."
Salida:  "help cant login"
```

Paso 4. Tokenización

Se utiliza nltk.word_tokenize() para dividir el texto en tokens individuales.

```
Entrada: "help cant login"
Salida:  ["help", "cant", "login"]
```

Paso 5. Filtrado de stopwords

Se eliminan palabras comunes del inglés (the, is, a, my, to, etc.) usando la lista de stopwords de NLTK. También se descartan tokens no alfabeticos.

```
Entrada: ["i", "need", "help", "with", "my", "order"]
Salida:  ["need", "help", "order"]
```

Paso 6. Stemming

Se aplica el SnowballStemmer (Porter2) de NLTK para reducir las palabras a su raíz morfológica.

```
"cancellation" -> "cancel"
"cancelled"    -> "cancel"
"shipping"     -> "ship"
"delivered"    -> "deliv"
```

4.2 Ejemplo Completo

```
Texto original: "I want to cancel my subscription {{Sub ID}}, please help!"

Paso 1 (placeholders): "I want to cancel my subscription , please help!"
Paso 2 (minúsculas):   "i want to cancel my subscription , please help!"
Paso 3 (puntuación):   "i want to cancel my subscription  please help"
Paso 4 (tokenización): ["i", "want", "to", "cancel", "my", "subscription", "please",
"help"]
Paso 5 (stopwords):    ["want", "cancel", "subscription", "please", "help"]
Paso 6 (stemming):     ["want", "cancel", "subscript", "pleas", "help"]
```

4.3 Herramientas Utilizadas

Herramienta	Uso	Justificación
re (Python stdlib)	Eliminación de placeholders	Expresiones regulares para patrones {{...}}
string (Python stdlib)	Eliminación de puntuación	Tabla de traduccion eficiente
nltk.word_tokenize	Tokenización	Tokenizador robusto que maneja contracciones
nltk.corpus.stopwords	Filtrado de stopwords	Lista curada de 179 stopwords en inglés
nltk.stem.SnowballStemmer	Stemming	Algoritmo Porter2, más preciso que Porter original

Nota: NLTK se utiliza únicamente para tokenización y stopwords, no para clasificación, conforme a las restricciones del proyecto.

4.4 Resultado del Preprocesamiento

Después de procesar los 26,872 documentos del dataset:

- Vocabulario resultante: 2,509 palabras únicas (tokens stemmeados).
- Total de tokens por clase: desde 4,200 (SUBSCRIPTION) hasta 24,287 (ACCOUNT).

5. Explicación Matemática del Algoritmo

5.1 Naive Bayes Multinomial

El clasificador Naive Bayes Multinomial calcula la probabilidad de que un documento pertenezca a cada clase y selecciona la clase con mayor probabilidad. Se basa en el Teorema de Bayes y la suposicion de independencia condicional entre palabras.

Teorema de Bayes:

```
P(clase | documento) = P(documento | clase) * P(clase) / P(documento)
```

Como P(documento) es constante para todas las clases, se simplifica a:

```
P(clase | documento) ~ P(clase) * P(documento | clase)
```

Con la suposicion de independencia (naive), la probabilidad del documento dado la clase se descompone como el producto de las probabilidades de cada palabra:

```
P(clase | documento) ~ P(clase) * PRODUCTO_i P(palabra_i | clase)
```

5.2 Probabilidad A Priori (Prior)

La probabilidad a priori de cada clase se calcula como la proporción de documentos que pertenecen a esa clase:

```
P(clase) = N_clase / N_total
```

Ejemplo con datos reales:

```
P(CANCEL) = 950 / 26,872   = 0.035353
P(ORDER)  = 3,988 / 26,872 = 0.148387
```

ORDER tiene un prior 4.2 veces mayor que CANCEL, lo que significa que antes de analizar el texto, el modelo ya considera más probable que un documento sea ORDER.

Tabla completa de probabilidades a priori:

Clase	N_clase	P(clase)
ACCOUNT	5,986	0.2228
ORDER	3,988	0.1484
REFUND	2,992	0.1113
CONTACT	1,999	0.0744
INVOICE	1,999	0.0744
PAYMENT	1,998	0.0743
FEEDBACK	1,997	0.0743
DELIVERY	1,994	0.0742
SHIPPING	1,970	0.0733
SUBSCRIPTION	999	0.0372
CANCEL	950	0.0354

5.3 Bag of Words (BoW)

El modelo utiliza la representación Bag of Words: cada documento se representa como un vector de frecuencias de palabras, ignorando el orden. Durante el entrenamiento, se construye un vocabulario con todas las palabras únicas del corpus preprocesado y se cuenta cuantas veces aparece cada palabra en cada clase.

Datos del modelo entrenado:

- Vocabulario: 2,509 palabras únicas.
- Para cada clase, se mantiene un contador de frecuencias. Por ejemplo, en la clase CANCEL: {"cancel": 182, "want": 109, "subscript": 87, ...}.

5.4 Laplace Smoothing

Problema: Si una palabra nunca aparece en una clase, su probabilidad condicional es cero:

```
P("order" | CANCEL) = 0 / 4,327 = 0
```

Al multiplicar probabilidades, un solo cero anularía toda la predicción:

```
P(CANCEL | doc) = P(CANCEL) * ... * P("order" | CANCEL) * ... = 0
```

Solución (Suavizado de Laplace): Se suma una constante alpha (= 1.0) al numerador y alpha * |V| al denominador:

```
P(palabra | clase) = (count(palabra, clase) + alpha) / (total_palabras_clase + alpha *
|V|)
```

Donde:

- count(palabra, clase) = veces que la palabra aparece en documentos de esa clase.
- total_palabras_clase = total de tokens en documentos de esa clase.
- alpha = 1.0 (add-one smoothing).

- |V| = 2,509 (tamano del vocabulario).

Ejemplo. Palabra presente: P("cancel" | CANCEL)

```
P("cancel" | CANCEL) = (182 + 1) / (4,327 + 1 * 2,509)
                     = 183 / 6,836
                     = 0.02677
```

Ejemplo. Palabra ausente: P("order" | CANCEL)

```
P("order" | CANCEL) = (0 + 1) / (4,327 + 1 * 2,509)
                    = 1 / 6,836
                    = 0.000146
```

Gracias al suavizado, la probabilidad es muy pequena pero no cero, permitiendo que el modelo siga funcionando.

5.5 Suma de Logaritmos (Log-Sum)

Problema: Multiplicar muchas probabilidades pequeñas causa underflow numérico (el resultado se redondea a cero en la computadora):

```
0.02677 * 0.000146 * 0.01593 * ... = 0.0000000... (underflow)
```

Solución: Usar logaritmos convierte las multiplicaciones en sumas:

```
log(a * b * c) = log(a) + log(b) + log(c)
```

La formula de inferencia se convierte en:

```
score(clase) = log P(clase) + SUM_i log P(palabra_i | clase)
```

Ejemplo completo. Clasificando "I want to cancel my order"

Tokens después de preprocesamiento: ["want", "cancel", "order"]

Cálculo para la clase CANCEL:

```
score(CANCEL) =   log P(CANCEL)
                + log P("want"   | CANCEL)
                + log P("cancel" | CANCEL)
                + log P("order"  | CANCEL)

              =   (-3.3424)
                + (-4.1295)
                + (-3.6205)
                + (-8.8300)

              = -19.9223
```

Cálculo para la clase ORDER:

```
score(ORDER)  =   log P(ORDER)
                + log P("want"   | ORDER)
                + log P("cancel" | ORDER)
                + log P("order"  | ORDER)

              =   (-1.9078)
                + (-3.5843)
                + (-3.0009)
                + (-2.3306)

              = -10.8236
```

Tabla de scores para las 11 clases:

Clase	log P(c)	+ "want"	+ "cancel"	+ "order"	Score
ORDER	-1.9078	-3.5843	-3.0009	-2.3306	-10.8236
SUBSCRIPTION	-3.2921	-3.9209	-3.3645	-8.8112	-19.3886
DELIVERY	-2.6009	-3.8828	-9.3122	-3.6918	-19.4877
CANCEL	-3.3424	-4.1295	-3.6205	-8.8300	-19.9223
ACCOUNT	-1.5017	-3.8074	-4.6469	-10.1960	-20.1520
REFUND	-2.1951	-3.8624	-9.6276	-6.7944	-22.4795
INVOICE	-2.5984	-3.9278	-9.1481	-9.1481	-24.8225
PAYMENT	-2.5989	-3.6224	-9.4246	-9.4246	-25.0705
FEEDBACK	-2.5994	-3.7634	-9.3581	-9.3581	-25.0789
CONTACT	-2.5984	-3.9049	-9.3342	-9.3342	-25.1718
SHIPPING	-2.6131	-3.8706	-9.3595	-9.3595	-25.2027

Selección (Argmax): La clase con el score menos negativo (más alto) gana:

```
clase_predicha = argmax(scores) = ORDER (-10.82)
```

5.6 Conversión a Confianza (Softmax)

Para obtener probabilidades interpretables que sumen 1.0, se aplica la función softmax con estabilizacion numérica:

```
Paso 1: Restar el máximo para estabilidad numérica
        max(scores) = -10.8236

Paso 2: Calcular exponenciales
        exp(-10.8236 - (-10.8236)) = exp(0)       = 1.000000  (ORDER)
        exp(-19.3886 - (-10.8236)) = exp(-8.5650) = 0.000191  (SUBSCRIPTION)
        exp(-19.9223 - (-10.8236)) = exp(-9.0987) = 0.000112  (CANCEL)
        ...demas clases ~ 0.000001
```

```
Paso 3: Normalizar (dividir entre la suma)
        confianza(ORDER)  = 1.000000 / 1.000576 = 0.9994 (99.94%)
        confianza(CANCEL) = 0.000112 / 1.000576 = 0.0001 ( 0.01%)
```

6. Métricas de Evaluación

6.1 Definiciones y Fórmulas

Para cada clase se calculan True Positives (TP), False Positives (FP) y False Negatives (FN):

- TP(c): documentos correctamente clasificados como clase c.
- FP(c): documentos incorrectamente clasificados como clase c (pertenecen a otra clase).
- FN(c): documentos de clase c incorrectamente clasificados como otra clase.

Precisión:

```
Precisión(c) = TP(c) / (TP(c) + FP(c))
```

Mide: de todas las predicciones de esta clase, cuantas fueron correctas.

Recall (Sensibilidad):

```
Recall(c) = TP(c) / (TP(c) + FN(c))
```

Mide: de todos los documentos reales de esta clase, cuantos fueron encontrados.

F1-Score:

```
F1(c) = 2 * (Precisión(c) * Recall(c)) / (Precisión(c) + Recall(c))
```

Media armónica entre precisión y recall. Penaliza valores desbalanceados.

Accuracy Global:

```
Accuracy = Total de predicciones correctas / Total de documentos
```

Macro F1:

```
Macro F1 = (1/C) * SUM_c F1(c)
```

Promedio simple del F1 de todas las clases, sin ponderar por tamano. Da igual importancia a clases pequeñas y grandes.

6.2 Resultados Obtenidos

Métricas globales (evaluadas sobre el dataset completo de 26,872 documentos):

Métrica	Valor
Accuracy	0.9929 (99.29%)
Macro F1	0.9928 (99.28%)

Métricas por clase:

Clase	Precisión	Recall	F1-Score	Support
ACCOUNT	0.9975	0.9987	0.9981	5,986
CANCEL	1.0000	0.9989	0.9995	950
CONTACT	1.0000	0.9990	0.9995	1,999
DELIVERY	0.9853	0.9388	0.9615	1,994
FEEDBACK	0.9960	0.9995	0.9978	1,997
INVOICE	0.9990	0.9950	0.9970	1,999
ORDER	0.9819	0.9922	0.9870	3,988
PAYMENT	0.9970	0.9985	0.9977	1,998
REFUND	1.0000	0.9967	0.9983	2,992
SHIPPING	0.9714	0.9990	0.9850	1,970
SUBSCRIPTION	0.9990	0.9990	0.9990	999

6.3 Matriz de Confusión

La matriz de confusión muestra, para cada clase real (filas), cuantos documentos fueron clasificados en cada clase predicha (columnas). La diagonal contiene las predicciones correctas.

Matriz de confusión completa (11 x 11):

Real / Pred	ACC	CAN	CON	DEL	FEE	INV	ORD	PAY	REF	SHI	SUB
ACCOUNT	5978	0	0	0	0	0	1	6	0	0	1
CANCEL	0	949	0	1	0	0	0	0	0	0	0
CONTACT	0	0	1997	0	2	0	0	0	0	0	0
DELIVERY	0	0	0	1872	0	0	69	0	0	53	0
FEEDBACK	0	0	0	0	1996	0	0	0	0	1	0
INVOICE	0	0	0	4	5	1989	0	0	0	1	0
ORDER	12	0	0	13	1	2	3957	0	0	3	0
PAYMENT	2	0	0	1	0	0	0	1995	0	0	0
REFUND	0	0	0	9	0	0	1	0	2982	0	0
SHIPPING	1	0	0	0	0	0	1	0	0	1968	0
SUBSCRIPTION	0	0	0	0	0	0	1	0	0	0	998

6.4 Análisis Crítico de la Matriz de Confusión

Clases con mejor rendimiento:

1. CANCEL (F1 = 0.9995, Precisión = 1.00): solo 1 error de 950 documentos. El vocabulario de cancelaciones es muy específico (terminate, cancel, stop billing).
2. CONTACT (F1 = 0.9995, Precisión = 1.00): solo 2 errores. Las solicitudes de contacto tienen vocabulario único (phone number, email address, reach).
3. REFUND (F1 = 0.9983, Precisión = 1.00): las solicitudes de reembolso contienen palabras muy discriminativas (refund, money back, reimburse).

Clases con menor rendimiento:

1. DELIVERY (F1 = 0.9615, Recall = 0.9388): la clase más débil. De 1,994 documentos de DELIVERY, 69 fueron clasificados como ORDER porque solicitudes de entrega frecuentemente mencionan "order" (my order hasn't arrived, order delivery status). 53 fueron clasificados como SHIPPING porque DELIVERY y SHIPPING comparten vocabulario semántico (package, arrive, tracking). El recall de 93.88% indica que el modelo falla en reconocer el 6.12% de los documentos de DELIVERY.
2. SHIPPING (F1 = 0.9850, Precisión = 0.9714): la precisión baja se debe a que absorbe errores de DELIVERY (53 documentos de DELIVERY clasificados como SHIPPING). El modelo confunde envio (SHIPPING) con entrega (DELIVERY) porque comparten contexto logístico.
3. ORDER (F1 = 0.9870, Precisión = 0.9819): recibe errores de DELIVERY (69 documentos) porque muchas solicitudes de entrega mencionan la palabra "order".

Patron principal de confusión: el triangulo DELIVERY-SHIPPING-ORDER concentra la mayoría de errores del modelo. Esto se explica porque las tres categorías pertenecen al mismo dominio logístico y comparten vocabulario significativo. Un clasificador Bag of Words ignora el contexto y el orden de las palabras, lo que dificulta distinguir entre "my order shipment" (SHIPPING) y "my order status" (ORDER).

7. Resultados de K-Folds Cross Validation

7.1 Metodología

Se implementó validación cruzada K-Fold de forma manual (sin scikit-learn) con los siguientes parámetros:

- K = 5 particiones.
- Seed = 42 para reproducibilidad.
- Tamano de cada fold: aproximadamente 5,374 documentos.
- Datos de entrenamiento por fold: aproximadamente 21,498 documentos.
- Datos de prueba por fold: aproximadamente 5,374 documentos.

Proceso:

```
1. Mezclar los 26,872 documentos aleatoriamente (seed=42)
2. Dividir en 5 particiones de tamano aproximadamente igual
3. Para cada fold i (de 1 a 5):
   a. Usar fold i como conjunto de prueba
   b. Usar los otros 4 folds como conjunto de entrenamiento
   c. Entrenar un modelo nuevo desde cero
   d. Predecir en el fold de prueba
   e. Calcular accuracy y macro F1
4. Promediar las métricas de los 5 folds
```

```
Fold 1: [TEST ][TRAIN][TRAIN][TRAIN][TRAIN]
Fold 2: [TRAIN][TEST ][TRAIN][TRAIN][TRAIN]
Fold 3: [TRAIN][TRAIN][TEST ][TRAIN][TRAIN]
Fold 4: [TRAIN][TRAIN][TRAIN][TEST ][TRAIN]
Fold 5: [TRAIN][TRAIN][TRAIN][TRAIN][TEST ]
```

7.2 Resultados por Fold

Fold	Accuracy	Macro F1
1	0.9914	0.9913
2	0.9924	0.9921
3	0.9916	0.9914
4	0.9905	0.9904
5	0.9885	0.9885

7.3 Estadísticas Agregadas

Métrica	Media	Desviación Estándar
Accuracy	0.9909 (99.09%)	0.0013 (0.13%)
Macro F1	0.9907 (99.07%)	0.0012 (0.12%)

7.4 Análisis de Varianza

La desviación estándar de 0.13% en accuracy y 0.12% en Macro F1 es extremadamente baja, lo que indica:

1. Estabilidad del modelo: el rendimiento no depende significativamente de que datos caen en entrenamiento vs prueba. El modelo generaliza consistentemente.
2. Ausencia de overfitting severo: la diferencia entre la accuracy en datos completos (99.29%) y la accuracy promedio de K-Fold (99.09%) es de solo 0.20 puntos porcentuales. Esto sugiere que el modelo no se ha sobreajustado significativamente al conjunto de entrenamiento.
3. Rango estrecho: el fold con mejor rendimiento (Fold 2: 99.24%) y el peor (Fold 5: 98.85%) difieren en solo 0.39 puntos porcentuales, confirmando la consistencia.
4. Dataset bien distribuido: la baja varianza entre folds también refleja que el dataset Bitext tiene etiquetas consistentes y semánticamente coherentes, lo que facilita el aprendizaje del modelo.

8. Diagramas

8.1 Arquitectura de la Solución

La siguiente figura muestra la arquitectura general del sistema desplegado mediante Docker Compose. El sistema se compone de dos contenedores principales: el frontend (nginx sirviendo la SPA de Vue 3 y actuando como proxy reverso) y el backend (FastAPI con el motor de inferencia Naive Bayes cargado en memoria y una base de datos SQLite para persistencia).

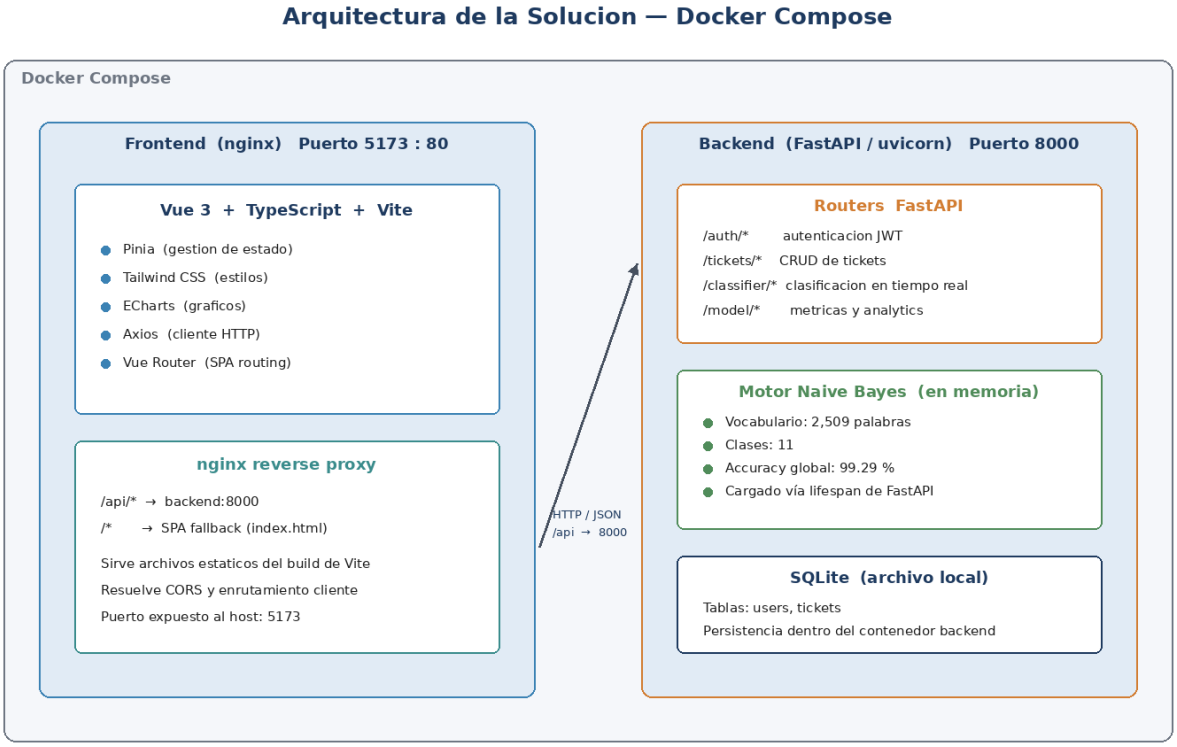


Figura 1. Arquitectura de la solución sobre Docker Compose.

Componentes:

Componente	Tecnología	Función
Frontend	Vue 3.5 + TypeScript 6 + Vite 8	Interfaz de usuario, SPA
Servidor web	nginx 1.27	Servir SPA, proxy reverso a backend
Backend	Python 3.11 + FastAPI 0.115	API REST, lógica de negocio
Motor de inferencia	Naive Bayes (from scratch)	Clasificación automática
Base de datos	SQLite (stdlib sqlite3)	Persistencia de tickets y usuarios
Orquestación	Docker Compose	Despliegue de ambos servicios

8.2 Diagrama de Casos de Uso

El siguiente diagrama UML describe los casos de uso principales del sistema. Se definen tres actores con distintos niveles de acceso: Cliente, Agente y Administrador. Cada actor tiene asociado un conjunto de casos de uso que corresponden a las acciones que puede realizar en el sistema.

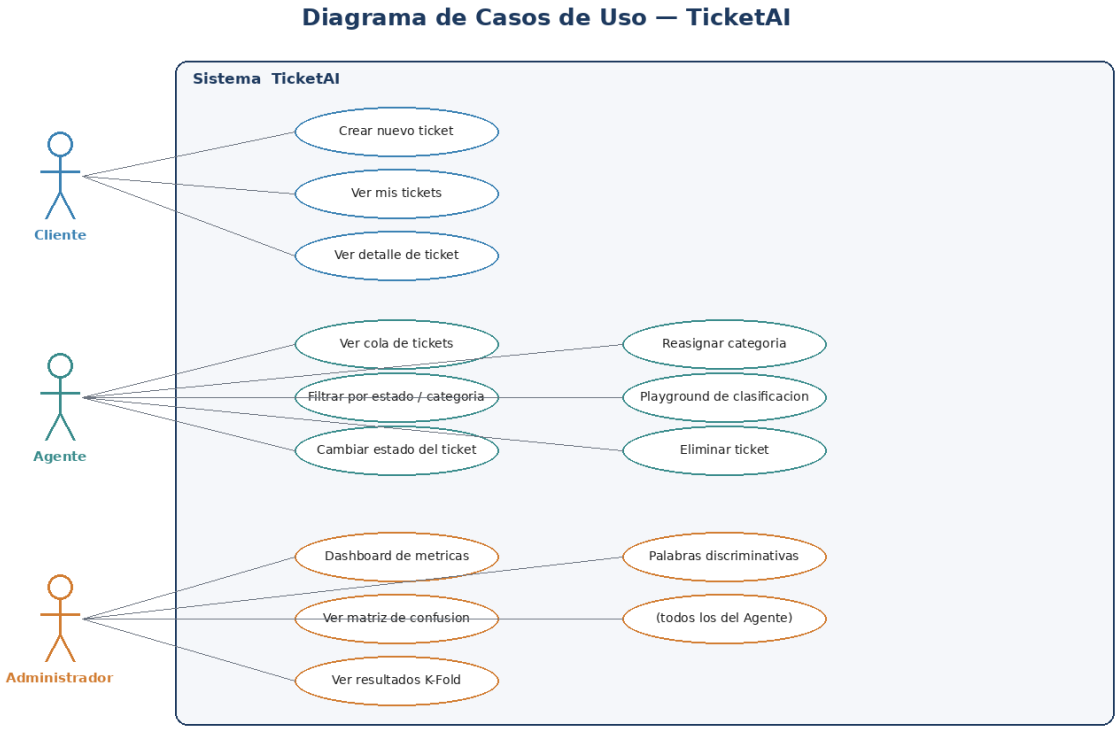


Figura 2. Diagrama UML de casos de uso de TicketAI.

Resumen de casos de uso por actor:

- Cliente: crear nuevo ticket, ver mis tickets, ver detalle de ticket.
- Agente: ver cola de tickets, filtrar por estado y categoría, cambiar estado, reasignar categoría, eliminar ticket, usar playground de clasificación.
- Administrador: todos los casos del Agente, además dashboard de métricas, ver matriz de confusión, ver resultados de K-Fold, ver palabras más discriminativas por categoría.

8.3 Diagrama de Flujo General

El siguiente diagrama muestra el flujo completo desde que el usuario ingresa un ticket hasta que recibe el resultado de la clasificación automática. Incluye las etapas de comunicación entre capas (frontend, proxy, backend), el procesamiento del texto, la inferencia del modelo y la persistencia en la base de datos.

Figura 3. Flujo general de creación y clasificación de un ticket.

8.4 Diagrama de Componentes

El siguiente diagrama de componentes describe la organizacion interna del frontend y el backend. En el frontend, la aplicación se estructura en features (Auth, Tickets, Classifier, Metrics) que comparten un cliente HTTP centralizado y un sistema de diseño comun. En el backend, los routers de FastAPI delegan la lógica en servicios especializados, y el motor de Naive Bayes opera como un módulo independiente integrado a traves del ciclo de vida de la aplicación.

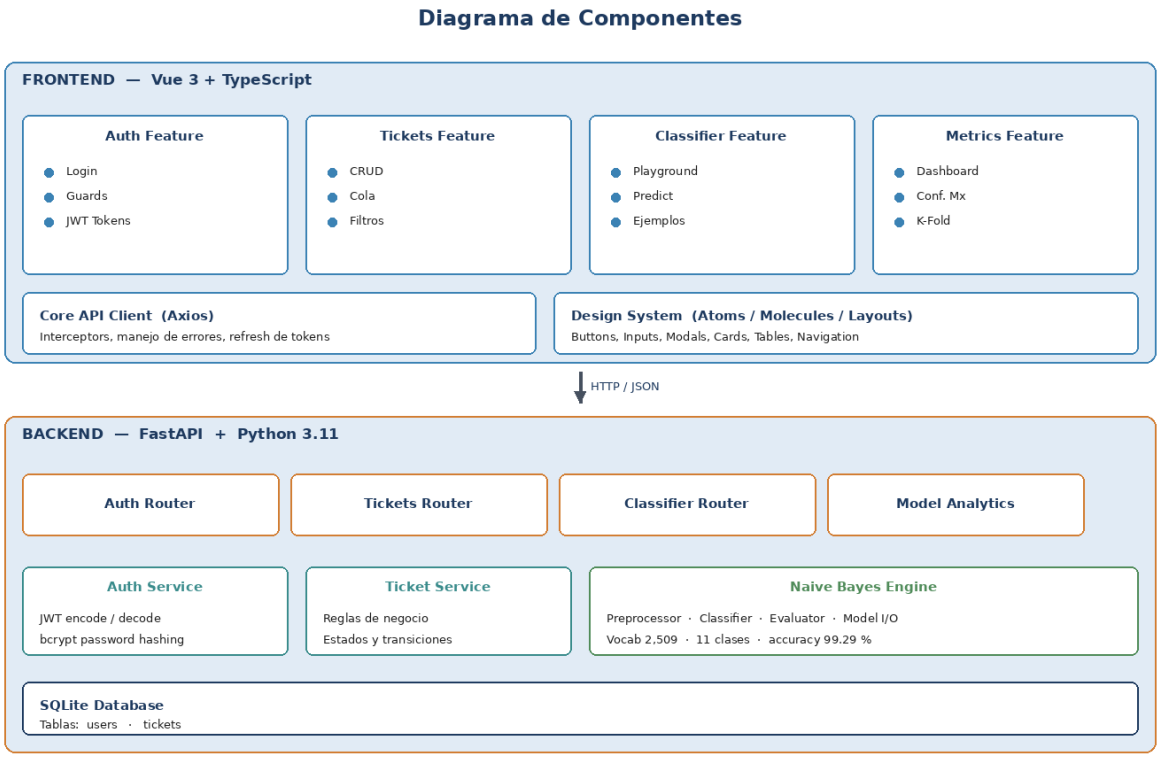


Figura 4. Diagrama de componentes del sistema.

8.5 Diagrama de Secuencias. Clasificación de Ticket

El siguiente diagrama de secuencias UML describe paso a paso la interacción entre los componentes del sistema cuando un usuario crea un ticket y recibe su clasificación automática. Las lineas continuas representan llamadas sincronas y las lineas discontinuas representan las respuestas.

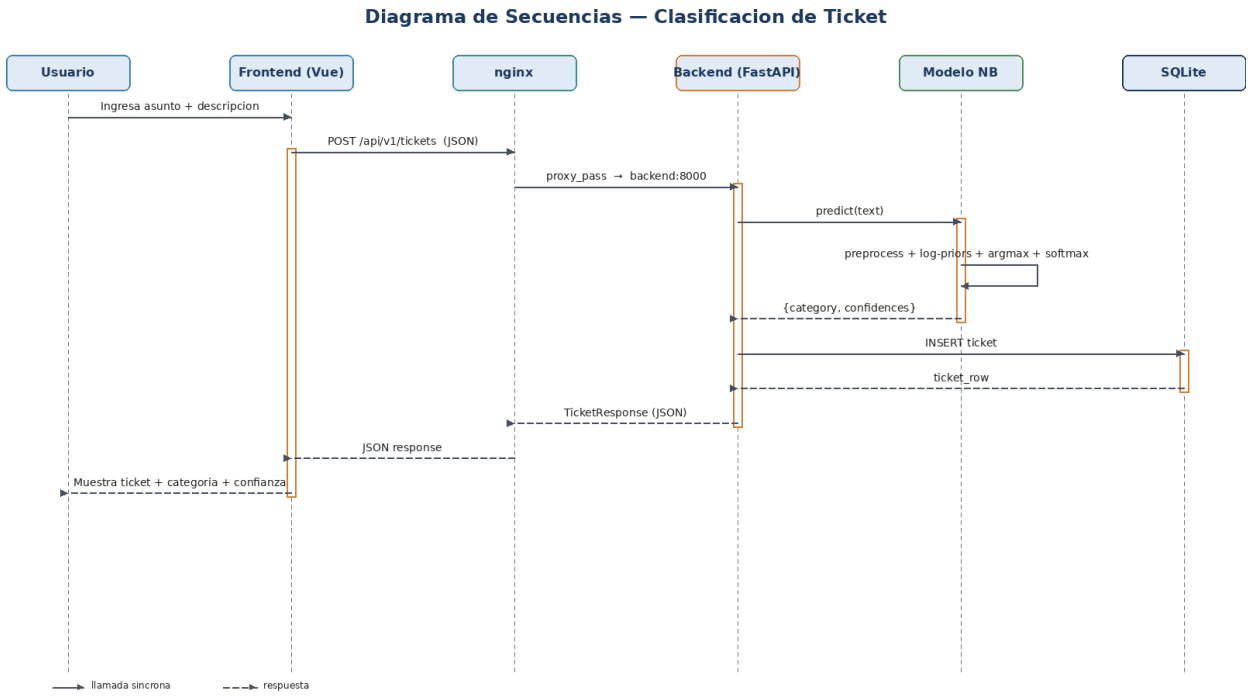


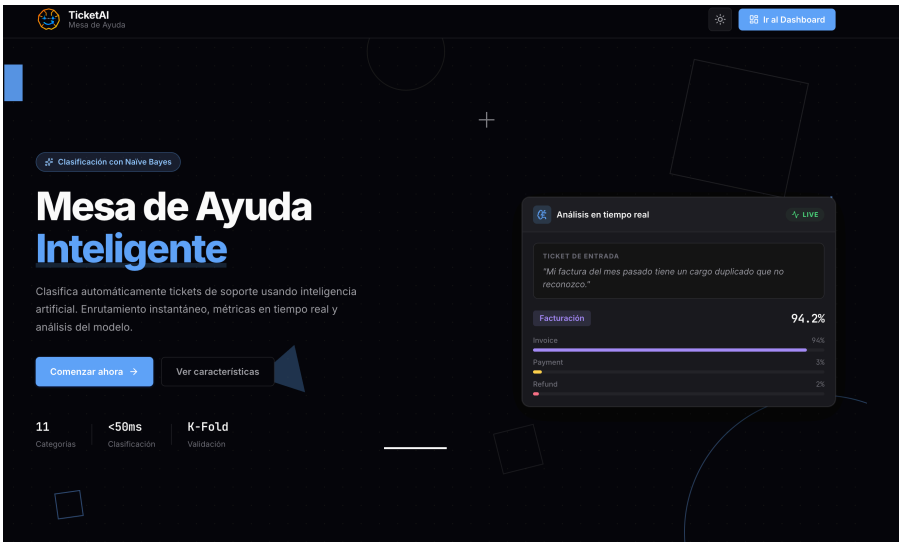
Figura 5. Diagrama de secuencias para la clasificación de un ticket.

9. Evidencias de Funcionamiento

En esta sección se incluyen las capturas de pantalla del sistema en funcionamiento. Cada evidencia corresponde a una vista especifica de la aplicación web desarrollada. Los marcadores indican el lugar donde cada captura debe ser insertada en la versión final del documento.

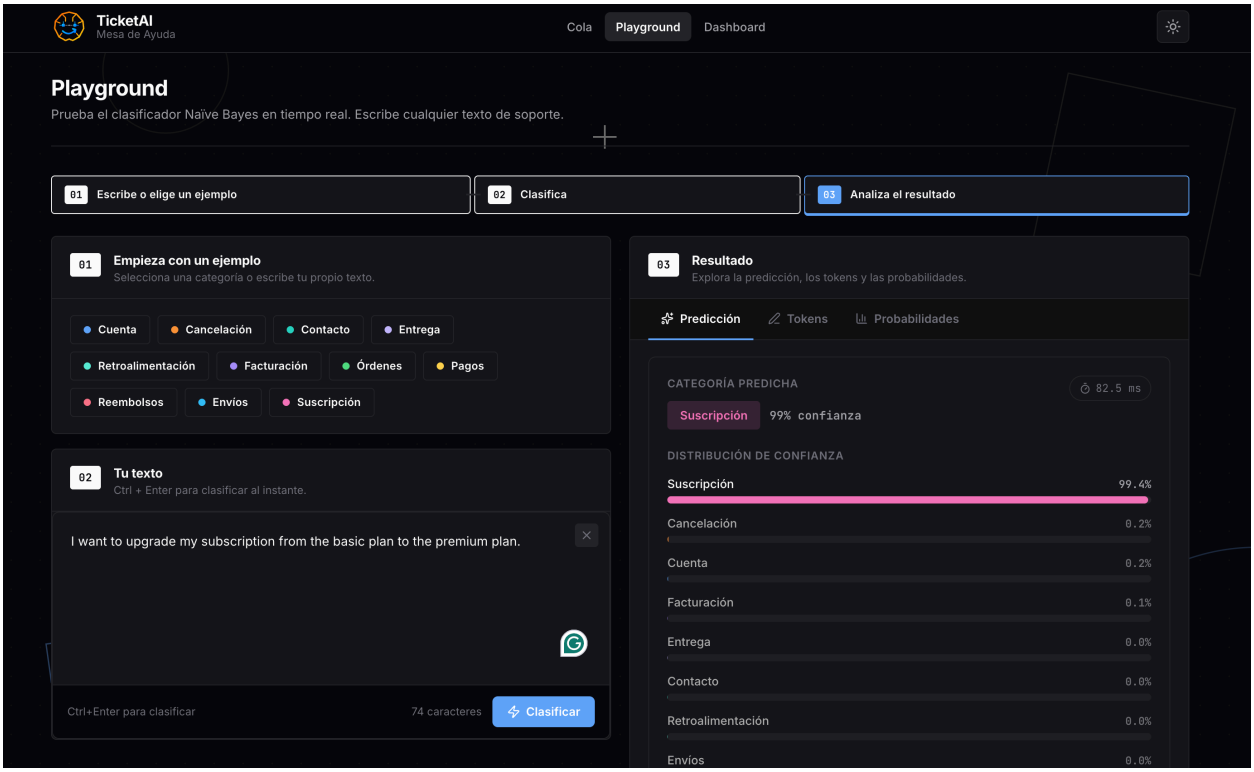
9.1 Página de inicio (Landing Page)

Página pública de bienvenida al sistema TicketAI con botón para ingresar.



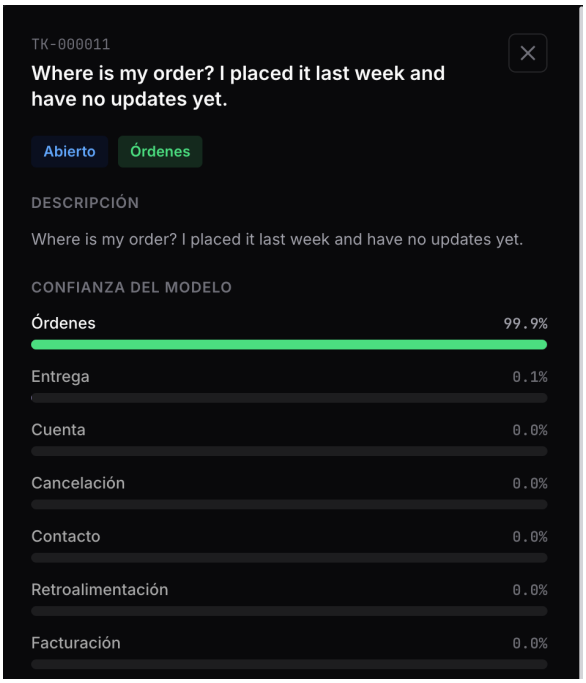
9.2 Creación de ticket

Formulario para ingresar descripción del ticket. Al enviar, el sistema clasifica automáticamente el ticket.



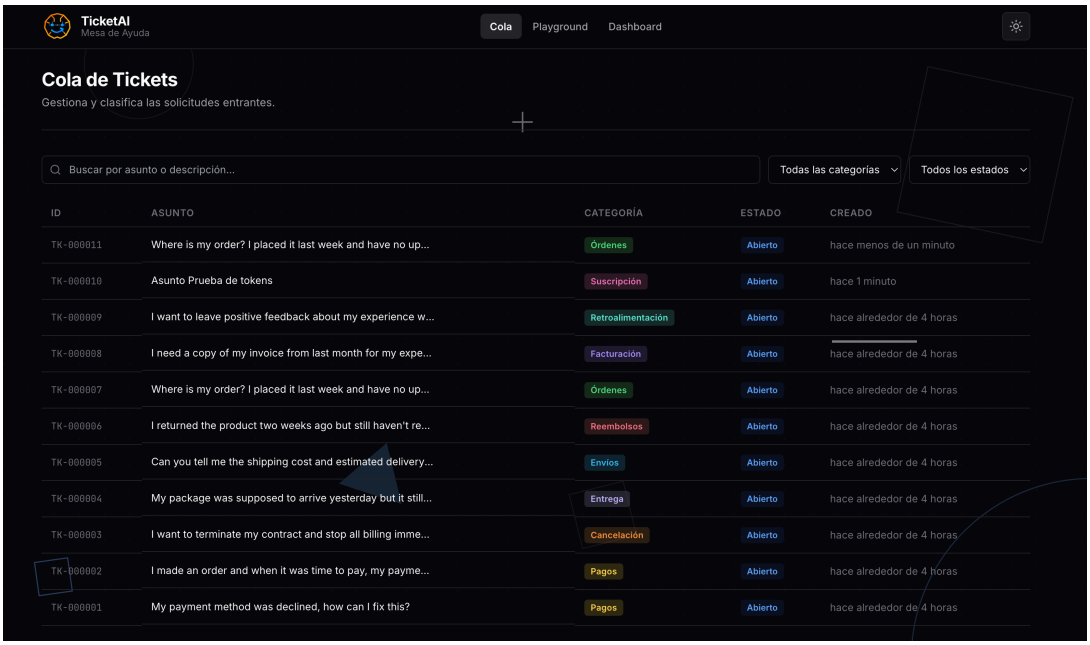
9.3 Resultado de clasificación

Ticket creado mostrando la categoría predicha (ej: ORDER), la barra de confianza con los porcentajes por categoría, el ID autogenerado y el estado "abierto".



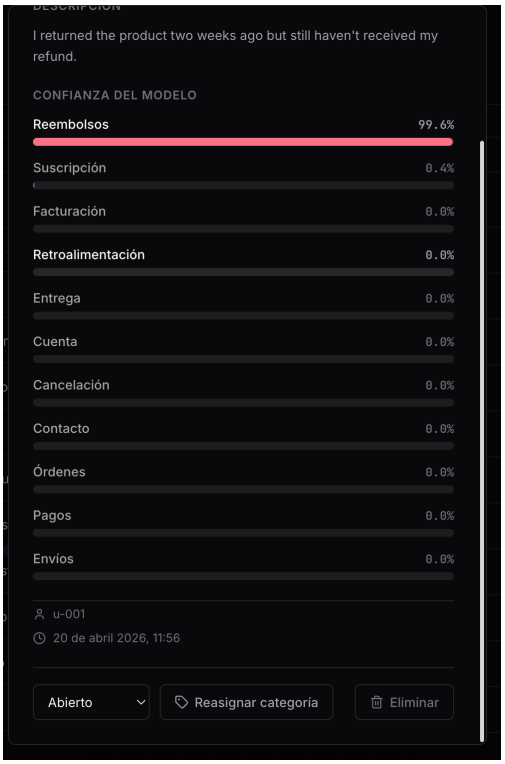
9.4 Cola de tickets

Tabla con todos los tickets del sistema, filtros por estado y categoría, búsqueda por texto, y paginación.



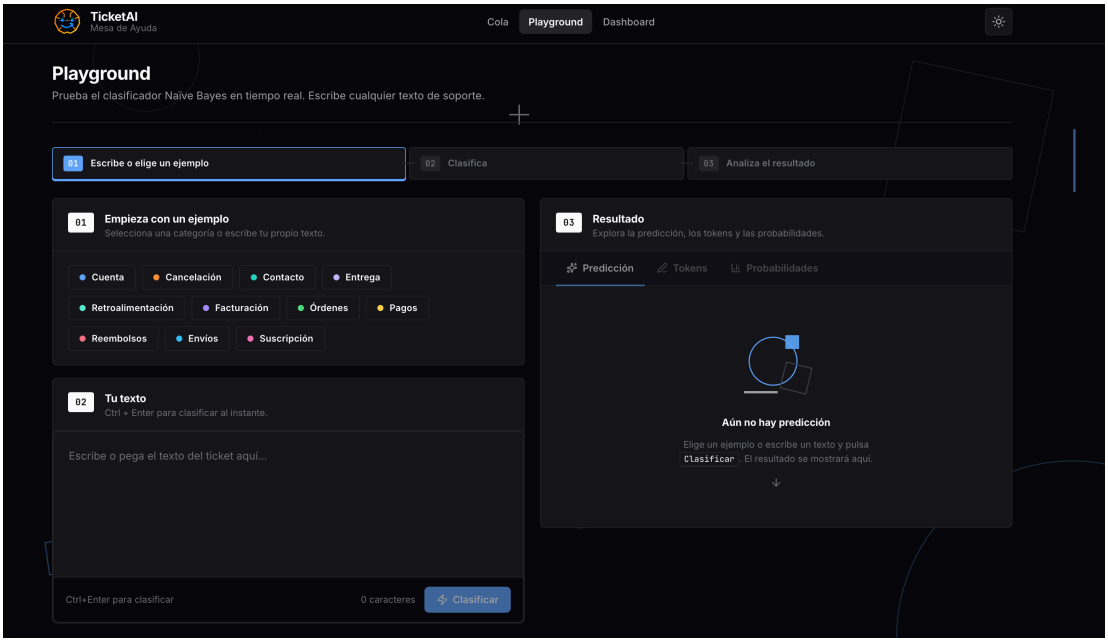
9.5 Detalle de ticket

Ventana emergente con el detalle completo del ticket: ID, asunto, descripción, categoría predicha, barra de confianza del modelo, información del creador, fecha, y acciones (cambiar estado, reasignar categoría, eliminar).



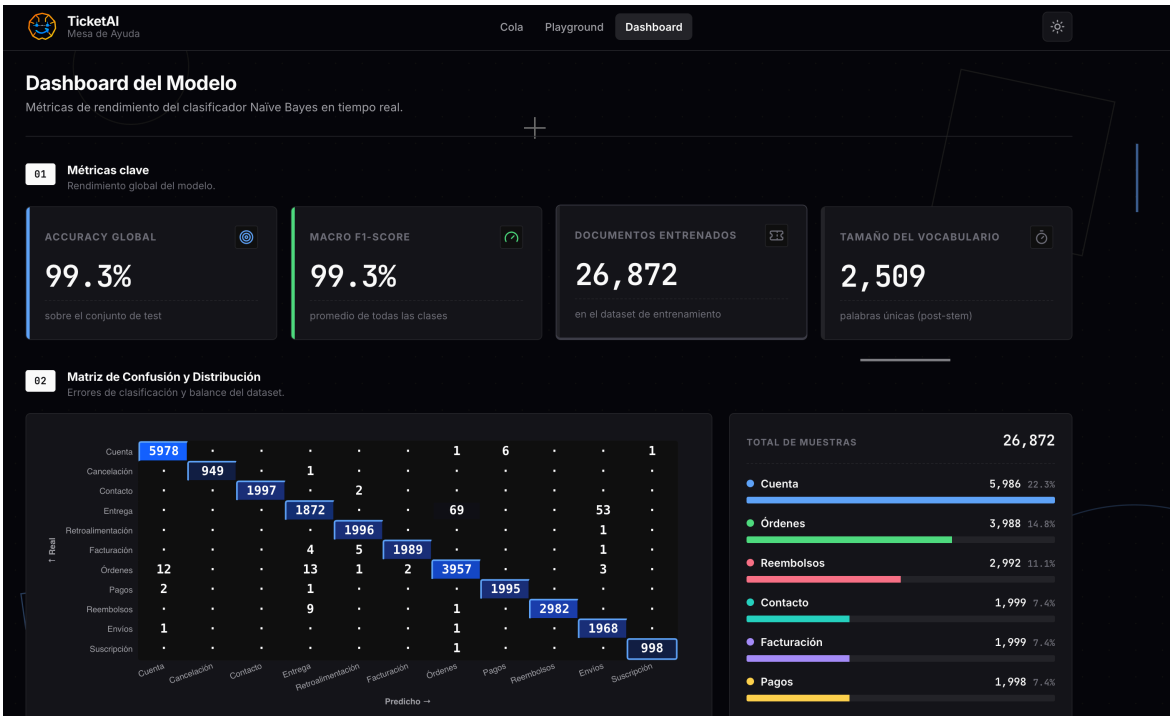
9.7 Playground de clasificación

Interfaz donde se puede escribir texto libre y obtener la clasificación en tiempo real con las confianzas por categoría. Incluye ejemplos predefinidos para cada categoría y botón para guardar como ticket.



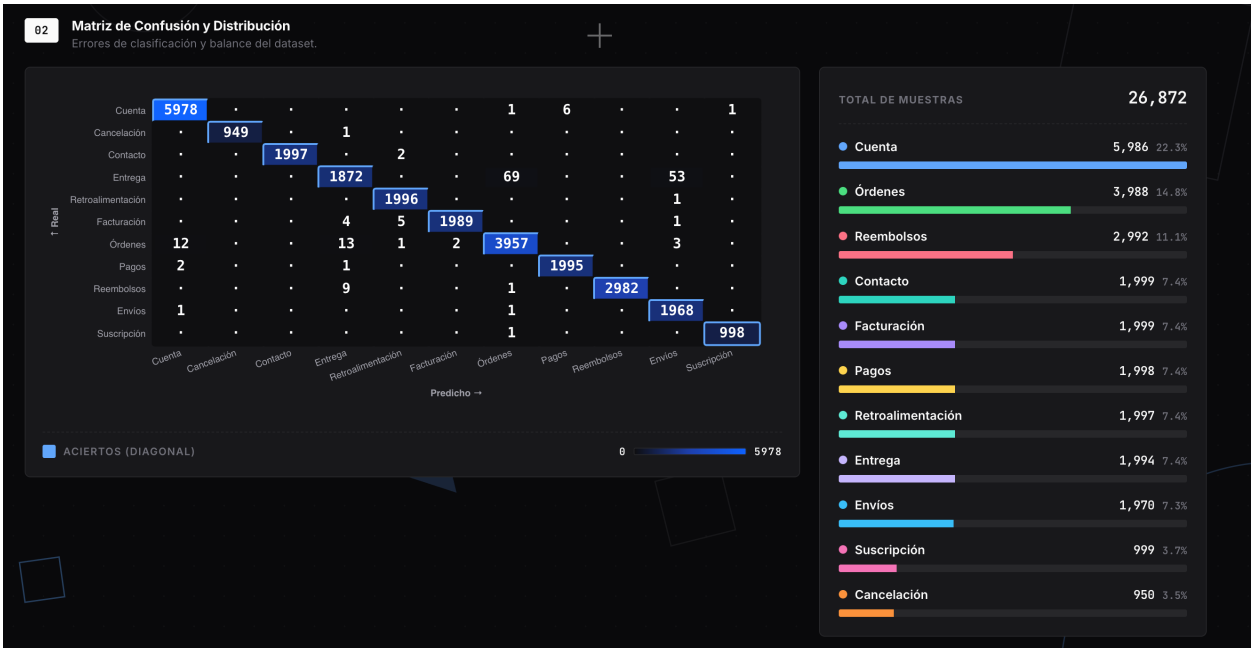
9.8 Dashboard de métricas

Panel mostrando información del modelo (versión, vocabulario, fecha de entrenamiento), accuracy global y Macro F1, métricas por clase (precisión, recall, F1), gráfico de matriz de confusión interactivo (ECharts) y resultados de K-Fold cross validation.



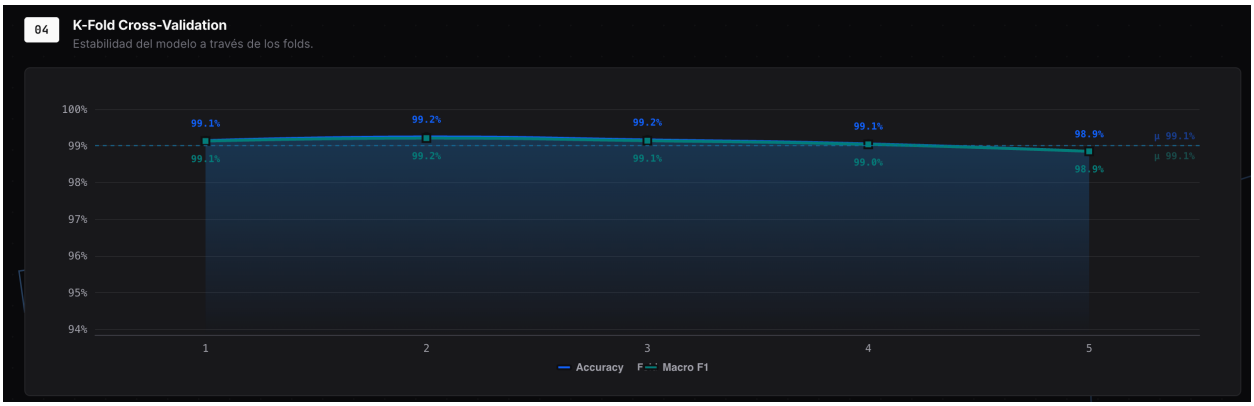
9.9 Matriz de confusión

Heatmap interactivo de 11x11 mostrando la distribución de predicciones correctas e incorrectas por clase. Los colores más intensos representan mayor concentracion de predicciones.



9.10 Resultados K-Fold (Gráfico)

Gráfico mostrando la accuracy y Macro F1 de cada uno de los 5 folds, con las lineas de media y desviación estándar.



10. Conclusiones y Aprendizajes

10.1 Conclusiones

1. Viabilidad del Naive Bayes desde cero: se demostró que es posible implementar un clasificador de texto funcional y preciso sin depender de librerias de machine learning. El modelo Naive Bayes Multinomial, implementado únicamente con Python stdlib, NLTK (tokenización) y numpy, alcanzo una accuracy del 99.29% en el dataset completo y 99.09% en validación cruzada K-Fold.
2. Importancia del preprocesamiento: el pipeline de 6 pasos (eliminación de placeholders, minúsculas, puntuación, tokenización, stopwords y stemming) fue clave para reducir el vocabulario de decenas de miles de palabras a 2,509 tokens significativos. El stemming en particular permitio agrupar variantes de la misma palabra, mejorando la capacidad del modelo para generalizar.
3. Efectividad del Laplace Smoothing: sin suavizado, el modelo fallaria con cualquier palabra no vista durante el entrenamiento. El suavizado de Laplace (alpha=1.0) garantizo que todas las palabras tuvieran una probabilidad no-cero, permitiendo clasificar textos con vocabulario nuevo.
4. Necesidad del log-sum: la suma de logaritmos fue esencial para evitar underflow numérico. Con 2,509 palabras en el vocabulario, multiplicar probabilidades pequeñas (del orden de 10^-4) habría producido valores indistinguibles de cero en la representación de punto flotante.
5. Limitaciones del modelo Bag of Words: la principal debilidad observada es la confusión entre DELIVERY, SHIPPING y ORDER, categorías que comparten vocabulario logístico. Un modelo Bag of Words ignora el contexto y el orden de las palabras, lo que limita su capacidad para distinguir intenciones cuando el vocabulario se traslapa.
6. Calidad del dataset: el dataset Bitext, con etiquetas generadas de forma controlada y semánticamente coherentes, fue fundamental para el alto rendimiento. La baja varianza entre folds (desviación estándar de 0.13%) confirma la consistencia de las etiquetas.

10.2 Aprendizajes

1. Comprension profunda del algoritmo: implementar Naive Bayes desde cero, paso a paso, forzo una comprension que no se obtiene al llamar sklearn.fit(). Entender por qué se necesita Laplace Smoothing, por qué se usan logaritmos, y como funciona softmax a nivel matematico proporciono una base sólida.
2. Importancia de la evaluación rigurosa: K-Fold cross validation demostró ser más informativo que una simple division train/test. Al entrenar 5 modelos independientes y promediar métricas, se obtuvo una estimacion confiable del rendimiento real del modelo, no solo su ajuste a un conjunto particular de datos.
3. Interpretacion de la matriz de confusión: la matriz de confusión revelo patrones de error que la accuracy global oculta. Mientras que 99.29% de accuracy parece perfecto, la matriz mostro que DELIVERY tiene un recall de solo 93.88%, un detalle invisible si solo se mira el número global.

4. Integración full-stack: la integracion entre el motor de inferencia (Python/FastAPI) y la aplicación web (Vue 3) a traves de Docker Compose demostró la importancia de definir contratos API claros (Pydantic schemas en backend, TypeScript interfaces en frontend) y de manejar correctamente la comunicación entre servicios (nginx reverse proxy, CORS).
5. El impacto del desbalance de datos: clases con más documentos (ACCOUNT: 5,986) tienen priors más altos y pueden dominar la clasificación de textos ambiguos. Esto se evidencio cuando textos con la palabra "cancel" eran clasificados como ORDER (3,988 docs) en lugar de CANCEL (950 docs), porque "cancel" aparece más veces en ORDER por su mayor volumen.

Anexo A. Stack Tecnológico Completo

Frontend

Tecnología	Versión	Uso
Vue	3.5	Framework reactivo
TypeScript	6.0	Tipado estático
Vite	8.0	Build tool y dev server
Pinia	3.0	Gestión de estado
Vue Router	4.6	Enrutamiento SPA
Tailwind CSS	3.4	Framework de estilos
Axios	1.15	Cliente HTTP
Zod	3.25	Validación de schemas
VeeValidate	4.15	Validación de formularios
ECharts	6.0	Gráficos interactivos
lucide-vue-next	1.0	Iconografía

Backend

Tecnología	Versión	Uso
Python	3.11	Lenguaje principal
FastAPI	0.115	Framework web
uvicorn	0.34	Servidor ASGI
SQLite	stdlib	Base de datos
PyJWT	2.10	Tokens JWT
passlib + bcrypt	1.7	Hashing de contraseñas
NLTK	3.9	Tokenización y stopwords
numpy	2.2	Operaciones numéricas
pandas	2.2	Carga de datos
pyarrow	19.0	Soporte Parquet
pytest	8.3	Testing (86 tests)

Infraestructura

Tecnología	Versión	Uso
Docker Compose	-	Orquestación de servicios
nginx	1.27	Proxy reverso + SPA
node	20 (alpine)	Build del frontend
python	3.11 (slim)	Imagen del backend

Anexo B. Guardado y Carga del Modelo

El modelo entrenado se serializa utilizando el módulo pickle de Python y se almacena en trained_models/model.pkl (aproximadamente 0.4 MB).

Guardado (después de entrenar):

```
import pickle

def save_model(model, path):
    with open(path, "wb") as f:
        pickle.dump(model, f)
```

Carga (al iniciar el servidor):

```
def load_model(path):
    with open(path, "rb") as f:
        return pickle.load(f)
```

Contenido del archivo serializado:

- Vocabulario completo: 2,509 palabras.
- Log-priors: 11 valores (uno por clase).
- Log-likelihoods: 11 x 2,509 = 27,599 valores.
- Conteos de palabras por clase.
- Hiperparámetro alpha = 1.0.
- Fecha de entrenamiento: 2026-04-18.

El modelo se carga automáticamente en memoria cuando el servidor FastAPI inicia (vía lifespan context manager) y permanece disponible para todas las predicciones durante la ejecución.

Anexo C. Instrucciones de Instalación y Ejecución

Opción 1. Docker Compose (recomendado)

```
# 1. Clonar repositorio
git clone <url-del-repositorio>
cd PROYECTO-IA-FRONTEND

# 2. Preparar modelo (una vez)
cd backend
pip install -r requirements.txt
python -m scripts.download_dataset
python -m scripts.train
cd ..

# 3. Levantar servicios
docker compose up --build

# 4. Acceder
# Frontend: http://localhost:5173
# Backend:  http://localhost:8000
# Health:   http://localhost:8000/health
```

Opción 2. Desarrollo local

```
# Terminal 1. Backend
cd backend
pip install -r requirements.txt
python -m scripts.download_dataset
python -m scripts.train
uvicorn app.main:app --reload --port 8000

# Terminal 2. Frontend
npm install
npm run dev
# Abrir http://localhost:5173
```

Ejecutar tests

```
cd backend
python -m pytest tests/ -v    # 86 tests
```